

Refonte complète du site de la Gym de Gimel

Je souhaite créer une nouvelle version du site de la Gym de Gimel, actuellement disponible à l'adresse suivante :

<https://www.gymel.ch/>

Le site actuel fonctionne avec WordPress et Divi. La nouvelle version doit être entièrement développée sur mesure avec Next.js. Elle ne doit pas simplement reproduire le site existant : elle doit proposer une expérience plus moderne, plus cohérente, plus rapide et plus facile à faire évoluer.

Le site WordPress actuel restera en ligne pendant tout le développement. Ne réalise aucune modification sur le site existant, son hébergement ou son nom de domaine.

1. Objectifs du projet

Les objectifs principaux sont les suivants :

- moderniser l'identité visuelle du site ;
- disposer d'une liberté graphique supérieure à WordPress et Divi ;
- rendre le site parfaitement adapté aux téléphones, tablettes et ordinateurs ;
- simplifier la maintenance du site ;
- créer des composants réutilisables ;
- automatiser l'affichage des concours, matchs, résultats et classements ;
- permettre à une personne non technique de mettre à jour les données sportives au moyen de fichiers CSV simples ;
- améliorer la visibilité des cours, inscriptions et prochains événements ;
- obtenir un projet professionnel propre, documenté, testable et présentable dans un portfolio.

Le site doit être en français.

2. Méthode de travail attendue

Avant de commencer à développer :

1. Inspecte l'ensemble du dépôt existant.
2. Identifie les technologies, fichiers et composants déjà présents.
3. Analyse la structure et les contenus du site actuel si celui-ci est accessible.
4. Ne copie pas simplement son apparence actuelle.
5. Dresse la liste des pages et contenus à conserver, modifier ou réorganiser.
6. Propose une architecture claire avant les changements importants.
7. Réutilise ce qui est pertinent dans le dépôt et évite les modifications inutiles.
8. Signale clairement les contenus, images ou informations qui manquent.

Si le site actuel n'est pas accessible depuis ton environnement, poursuis le développement avec des contenus temporaires réalistes et crée un document récapitulant les éléments qu'il faudra récupérer manuellement.

Travaille par étapes cohérentes. Après chaque étape importante, vérifie que le projet compile et que les fonctionnalités existantes ne sont pas cassées.

3. Technologies souhaitées

Utilise en priorité :

- Next.js avec App Router ;
- TypeScript en mode strict ;
- React ;
- Tailwind CSS ;
- des Server Components par défaut ;
- des Client Components uniquement lorsque l'interactivité le nécessite ;
- Zod pour la validation des données ;
- une bibliothèque fiable pour analyser les fichiers CSV ;
- les fonctionnalités natives de Next.js pour les images, les métadonnées, le cache et la revalidation.

Évite d'ajouter des dépendances volumineuses lorsqu'une solution simple suffit.

Le projet doit pouvoir être déployé sur Vercel.

4. Architecture générale

Organise le projet de façon claire, par exemple :

```
src/
├─ app/
├─ components/
│   ├─ layout/
│   ├─ ui/
│   ├─ courses/
│   ├─ events/
│   └─ sports/
├─ data/
├─ lib/
│   ├─ csv/
│   ├─ validation/
│   ├─ formatting/
│   └─ constants/
├─ types/
└─ styles/

public/
├─ images/
├─ documents/
└─ logos/

data/
```

```
├─ competitions.csv
├─ volleyball-men.csv
├─ volleyball-women.csv
├─ courses.csv
└─ closures.csv
```

Cette structure peut être adaptée si le dépôt possède déjà une organisation cohérente.

Crée également un fichier `AGENTS.md` à la racine du projet. Il doit documenter :

- l'objectif du site ;
- l'architecture ;
- les conventions de code ;
- les commandes à exécuter ;
- les règles de validation ;
- le fonctionnement des CSV ;
- les éléments à ne pas modifier ;
- la procédure à suivre avant de considérer une tâche comme terminée.

5. Direction visuelle

Le design doit correspondre à une société sportive locale, dynamique et accueillante.

Il doit être :

- moderne ;
- sportif sans être agressif ;
- accessible à toutes les générations ;
- lisible ;
- aéré ;
- chaleureux ;
- professionnel ;
- visuellement cohérent.

Utilise comme couleur principale le bleu suivant :

```
rgb(0, 44, 95)
```

Tu peux compléter cette couleur avec :

- du blanc ;
- des tons neutres clairs ;
- une couleur d'accent adaptée à l'identité existante ;
- des couleurs d'état accessibles pour les inscriptions, événements terminés, résultats et alertes.

Évite :

- les effets visuels excessifs ;
- les gradients génériques utilisés partout ;

- les animations inutiles ;
- les blocs surchargés ;
- les textes trop petits ;
- les mises en page ressemblant à un modèle générique généré automatiquement.

Prévois une hiérarchie typographique claire, des espacements cohérents et des composants avec des états de survol, de focus et d'erreur accessibles.

Utilise le logo, les photographies et les éléments graphiques présents dans le dépôt. Lorsque des images manquent, utilise des espaces réservés clairement identifiés, sans inventer de faux visuels officiels.

6. Navigation générale

Prévois une navigation claire avec les rubriques principales suivantes :

- Accueil ;
- Nos cours ;
- Calendrier sportif ;
- Événements ;
- Inscriptions ;
- La société ;
- Contact.

La navigation doit comprendre :

- un en-tête adapté aux écrans larges ;
- un menu mobile accessible ;
- un bouton d'inscription visible ;
- l'indication de la page active ;
- un pied de page complet ;
- les coordonnées utiles ;
- les liens vers les réseaux sociaux ;
- les informations légales nécessaires.

Le menu mobile doit être utilisable au clavier et compatible avec les lecteurs d'écran.

7. Page d'accueil

La page d'accueil doit permettre de comprendre immédiatement :

- qui est la Gym de Gimel ;
- quelles activités sont proposées ;
- comment trouver un cours ;
- comment s'inscrire ;
- quels événements arrivent prochainement.

Prévois au minimum les sections suivantes :

En-tête principal

- une photographie ou une composition visuelle forte ;
- un titre clair ;
- une courte présentation ;
- un bouton « Découvrir les cours » ;
- un bouton « S'inscrire ».

Prochains événements

Affiche automatiquement les prochains concours, matchs ou événements provenant des données disponibles.

Chaque carte doit pouvoir contenir :

- la date ;
- le titre ;
- le lieu ;
- la catégorie ;
- le statut ;
- un lien vers les détails.

Trouver un cours

Présente les cours les plus importants et propose un accès rapide à la liste complète.

Horaires ou activités de la semaine

Présente un aperçu lisible des cours selon les jours.

Présentation de la société

Ajoute une courte section consacrée à la Gym de Gimel, à son histoire et à ses valeurs.

Actualités ou informations importantes

Prévois une zone permettant de mettre en avant une inscription, une fermeture, une manifestation ou une communication importante.

Partenaires et sponsors

Les partenaires doivent être présents, mais ne doivent pas dominer la page d'accueil. Prévois un affichage cohérent et responsive des logos.

8. Gestion des cours

Les cours doivent être structurés comme des données et ne doivent pas être codés individuellement dans chaque page.

Chaque cours doit pouvoir contenir :

- un identifiant unique ;
- un slug ;
- un nom ;
- une courte description ;
- une description complète ;
- une catégorie ;
- une tranche d'âge ;
- un ou plusieurs jours ;
- une heure de début ;
- une heure de fin ;
- un lieu ;
- un ou plusieurs moniteurs ;
- un tarif ;
- un statut d'inscription ;
- une image ;
- un lien d'inscription ;
- une éventuelle remarque.

Prévois :

- une page listant tous les cours ;
- des filtres par catégorie, âge et jour ;
- une recherche simple si elle apporte une vraie valeur ;
- une page détaillée pour chaque cours ;
- une carte réutilisable pour les cours ;
- un message clair lorsqu'aucun cours ne correspond aux filtres.

Les filtres doivent fonctionner correctement sur mobile et être accessibles au clavier.

9. Calendrier sportif automatisé

La partie la plus importante du projet est le calendrier sportif.

Les données doivent être récupérées depuis des fichiers CSV stockés dans un dépôt GitHub. Les URL des fichiers ne doivent pas être codées directement à plusieurs endroits.

Utilise des variables d'environnement, par exemple :

```
COMPETITIONS_CSV_URL=  
VOLLEYBALL_MEN_CSV_URL=  
VOLLEYBALL_WOMEN_CSV_URL=  
CSV_REVALIDATE_SECONDS=300
```

Centralise leur utilisation dans un module de configuration.

Les fichiers CSV doivent être récupérés côté serveur. Ne les charge pas directement depuis le navigateur.

Utilise le système de cache et de revalidation de Next.js. La durée par défaut peut être de cinq minutes, mais elle doit pouvoir être configurée.

Concours de gymnastique

Prévois un fichier `competitions.csv` avec une structure semblable à celle-ci :

```
id,slug,title,startDate,endDate,location,category,group,status,description,registrationUrl,pro
```

Valeurs possibles pour `status` :

```
draft
upcoming
registration-open
registration-closed
finished
cancelled
```

Chaque concours doit pouvoir apparaître :

- dans le calendrier sportif ;
- dans la liste des prochains événements ;
- sur la page d'accueil lorsqu'il est mis en avant ;
- sur une page détaillée générée automatiquement.

La page détaillée doit pouvoir présenter :

- le titre ;
- la date ;
- le lieu ;
- le groupe concerné ;
- la description ;
- le statut ;
- le lien d'inscription ;
- le programme ;
- les résultats ;
- les documents disponibles.

Volley masculin et féminin

Prévois deux fichiers séparés :

```
volleyball-men.csv
volleyball-women.csv
```

Structure proposée :

```
id,season,date,time,competition,homeTeam,awayTeam,location,homeScore,awayScore,status,external
```

Valeurs possibles pour `status` :

```
scheduled  
postponed  
cancelled  
finished
```

Le site doit permettre d'afficher :

- les prochains matchs ;
- les matchs terminés ;
- les résultats ;
- les matchs à domicile et à l'extérieur ;
- un filtre par équipe ou saison si plusieurs équipes sont ajoutées ;
- une séparation claire entre volley masculin et féminin.

Si les données disponibles permettent de calculer un classement fiable, prépare une architecture permettant de le faire. Ne crée toutefois pas un classement trompeur si les règles officielles ou toutes les rencontres ne sont pas disponibles.

Le classement pourra également être lu depuis un fichier CSV séparé si cela est plus fiable.

10. Validation et gestion des erreurs CSV

Crée des schémas Zod distincts pour chaque type de donnée.

La validation doit notamment vérifier :

- les identifiants obligatoires ;
- l'unicité des identifiants et des slugs ;
- les dates ;
- les heures ;
- les URL ;
- les statuts autorisés ;
- les scores ;
- les champs obligatoires ;
- les valeurs booléennes ;
- la cohérence entre une date de début et une date de fin.

Une ligne incorrecte ne doit pas faire tomber l'ensemble du site.

En cas d'erreur :

1. ignore uniquement la ligne incorrecte lorsque cela est possible ;
2. enregistre une erreur compréhensible côté serveur ;
3. précise le fichier et la ligne concernés ;

4. n'affiche jamais de message technique brut au visiteur ;
5. affiche un état vide ou un message adapté si aucune donnée valide n'est disponible.

Ajoute des fichiers CSV locaux de démonstration et de secours.

Lorsque la récupération distante échoue, utilise la stratégie suivante :

1. tenter de récupérer le CSV distant ;
2. valider son contenu ;
3. utiliser les données locales de secours si la récupération ou la validation globale échoue ;
4. afficher le site sans erreur fatale ;
5. journaliser clairement le problème.

Ne masque pas silencieusement les erreurs pendant le développement.

11. Interface du calendrier sportif

La page « Calendrier sportif » doit être claire et facile à parcourir.

Prévois les catégories suivantes :

- Tous ;
- Concours de gymnastique ;
- Volley féminin ;
- Volley masculin.

Prévois également :

- une vue en liste adaptée au mobile ;
- un regroupement par mois ;
- une distinction visuelle entre événements à venir et terminés ;
- des filtres accessibles ;
- des liens vers les détails ;
- des boutons vers les programmes, résultats et inscriptions lorsqu'ils existent ;
- un état vide soigné ;
- une gestion correcte des événements annulés ou reportés.

Une vue mensuelle graphique peut être ajoutée uniquement si elle reste réellement utilisable sur mobile. La vue en liste doit rester la vue principale ou être toujours disponible.

Les événements passés doivent être archivés automatiquement en fonction de leur date et de leur statut.

12. Mise à jour des contenus par une personne non technique

Le système doit être pensé pour qu'Adriano puisse mettre à jour les concours et les calendriers sans modifier le code.

Crée une documentation simple dans :

```
docs/gestion-contenu.md
```

Cette documentation doit expliquer :

- où se trouvent les fichiers CSV ;
- comment ouvrir un fichier dans GitHub ;
- comment ajouter une ligne ;
- comment modifier un événement ;
- comment indiquer un résultat ;
- comment ajouter un lien vers un PDF ;
- quelles valeurs sont autorisées ;
- comment éviter de supprimer les en-têtes ;
- comment écrire les dates et les heures ;
- comment vérifier que la modification est correcte ;
- que faire lorsqu'une erreur de validation est signalée.

Ajoute plusieurs exemples réalistes.

Crée également un modèle vierge pour chaque fichier CSV.

13. Validation automatique sur GitHub

Ajoute une GitHub Action qui s'exécute lorsque les fichiers CSV ou le code sont modifiés.

Elle doit au minimum exécuter :

```
lint
typecheck
tests
build
```

Ajoute également un script spécifique de validation des CSV, par exemple :

```
npm run validate:data
```

En cas d'erreur CSV, le message doit préciser :

- le fichier concerné ;
- le numéro de ligne ;
- la colonne concernée ;
- la valeur reçue ;
- la valeur ou le format attendu.

La validation doit empêcher l'intégration d'un fichier invalide lorsque cela est possible.

14. Pages supplémentaires

Prévois les pages suivantes :

Inscriptions

- explication du fonctionnement ;
- accès aux formulaires ;
- statut des inscriptions ;
- cotisations ;
- équipements ;
- démissions ;
- autorisation liée aux photographies ;
- fermetures annuelles.

La société

- présentation ;
- histoire ;
- valeurs ;
- comité ;
- moniteurs ;
- documents importants si nécessaire.

Événements

- liste des manifestations qui ne sont pas nécessairement des compétitions ;
- événements à venir ;
- archives ;
- pages détaillées si nécessaire.

Contact

- coordonnées ;
- adresse ;
- liens utiles ;
- formulaire ou lien de contact ;
- informations permettant de contacter la bonne personne.

N'envoie pas réellement d'e-mails tant qu'aucune solution d'envoi n'a été configurée. Prépare le formulaire et sa validation, mais documente la configuration nécessaire.

15. Responsive design

Le site doit être conçu en priorité pour les appareils mobiles.

Vérifie au minimum les affichages suivants :

- 320 px ;
- 375 px ;
- 768 px ;

- 1024 px ;
- 1440 px.

Évite :

- les débordements horizontaux ;
- les tableaux impossibles à lire ;
- les boutons trop petits ;
- les menus difficiles à fermer ;
- les titres coupés ;
- les cartes avec des hauteurs incohérentes ;
- les éléments interactifs uniquement utilisables à la souris.

Les calendriers et résultats doivent rester lisibles sur les petits écrans.

16. Accessibilité

Respecte autant que possible les bonnes pratiques WCAG 2.2 niveau AA.

Vérifie notamment :

- les contrastes ;
- la navigation au clavier ;
- les états de focus ;
- les titres hiérarchisés ;
- les labels de formulaires ;
- les textes alternatifs ;
- les boutons et liens correctement identifiés ;
- les messages d'erreur ;
- les menus mobiles ;
- les boîtes de dialogue ;
- le respect de `prefers-reduced-motion`.

N'utilise pas uniquement la couleur pour communiquer un statut.

17. Référencement et performances

Ajoute :

- des métadonnées par page ;
- des titres et descriptions uniques ;
- les balises Open Graph ;
- un sitemap ;
- un fichier robots ;
- des URL propres ;
- des données structurées lorsque cela est pertinent ;
- une page 404 personnalisée ;
- une gestion correcte des anciennes URL lors de la migration finale.

Optimise :

- les images avec `next/image` ;
- le chargement des polices ;
- le JavaScript côté client ;
- la taille des dépendances ;
- le chargement différé des éléments non essentiels ;
- le cache des données distantes.

Évite de transformer des pages statiques en Client Components sans nécessité.

18. Sécurité

Ne place aucun secret dans le code source.

Utilise des variables d'environnement pour :

- les URL privées éventuelles ;
- les jetons de revalidation ;
- les services externes ;
- les futures fonctionnalités d'administration.

Si une route de revalidation manuelle est créée, protège-la avec un secret et valide correctement la requête.

Échappe et valide les données provenant des CSV avant de les afficher.

Les CSV ne doivent pas permettre d'injecter du HTML ou du JavaScript arbitraire.

19. Tests

Ajoute des tests ciblés et utiles.

Teste au minimum :

- l'analyse des CSV ;
- la validation Zod ;
- les dates ;
- les statuts ;
- les données manquantes ;
- les lignes incorrectes ;
- les identifiants dupliqués ;
- l'utilisation des données locales de secours ;
- le tri chronologique ;
- la séparation entre événements passés et futurs ;
- le calcul des résultats lorsque celui-ci est présent.

Ajoute quelques tests d'interface ou de parcours pour vérifier :

- l'ouverture du menu mobile ;

- la navigation vers un cours ;
- les filtres du calendrier sportif ;
- l'affichage d'un événement ;
- les états vides.

20. Documentation technique

Complète le fichier `README.md` avec :

- une présentation du projet ;
- les prérequis ;
- l'installation ;
- les variables d'environnement ;
- les commandes disponibles ;
- le fonctionnement des CSV ;
- la procédure de développement ;
- la procédure de build ;
- la procédure de déploiement sur Vercel ;
- la stratégie de cache ;
- la gestion des erreurs ;
- la structure du projet.

Fournis un fichier :

```
.env.example
```

sans valeur sensible.

Ajoute également :

```
docs/
├─ architecture.md
├─ gestion-contenu.md
├─ migration-wordpress.md
└─ deployment.md
```

21. Migration depuis WordPress

Ne remplace pas encore le site WordPress.

Prépare un document de migration qui décrit :

- les pages existantes ;
- les contenus repris ;
- les contenus à vérifier ;
- les anciennes URL ;
- les nouvelles URL ;
- les redirections nécessaires ;

- les images à récupérer ;
- les documents à migrer ;
- les éléments obsolètes détectés ;
- les étapes de mise en ligne ;
- une procédure de retour arrière.

La mise en production ne devra être effectuée qu'après validation humaine.

Ne modifie pas :

- les DNS ;
- le domaine ;
- l'hébergement WordPress ;
- les données du site actuel ;
- les comptes existants.

22. Première version attendue

Pour la première version fonctionnelle, concentre-toi sur :

1. la structure globale du projet ;
2. le système de design ;
3. l'en-tête et le pied de page ;
4. la page d'accueil ;
5. la liste des cours ;
6. une page détaillée de cours ;
7. le calendrier sportif ;
8. une page détaillée de concours ;
9. la récupération et la validation des CSV ;
10. les données locales de démonstration ;
11. la documentation de mise à jour ;
12. les tests essentiels.

Les pages secondaires peuvent être préparées avec une structure propre et des contenus provisoires, mais les parcours principaux doivent être complètement fonctionnels.

23. Critères d'acceptation

Le travail pourra être considéré comme terminé lorsque :

- le projet s'installe sans erreur ;
- le serveur de développement fonctionne ;
- le lint passe ;
- le typecheck passe ;
- les tests passent ;
- le build de production passe ;
- les CSV distants sont récupérés côté serveur ;
- les données sont validées ;
- une ligne CSV incorrecte ne fait pas tomber le site ;
- les fichiers locaux de secours fonctionnent ;
- les concours sont triés automatiquement ;

- les événements passés sont archivés ;
- les pages dynamiques sont générées correctement ;
- les filtres sont accessibles ;
- le site est utilisable sur mobile ;
- les principales pages possèdent leurs métadonnées ;
- aucune information sensible n'est présente dans le dépôt ;
- la documentation permet à une personne non technique de modifier les données.

24. Consignes finales

Ne crée pas uniquement une maquette visuelle. Produis une base fonctionnelle, maintenable et prête à évoluer.

Ne surdéveloppe pas la première version :

- pas de base de données ;
- pas de CMS personnalisé ;
- pas d'authentification ;
- pas d'interface d'administration ;
- pas de système complexe de permissions.

Les CSV GitHub constituent la source de données de la première version.

Lorsque tu dois faire un choix non précisé :

1. choisis la solution la plus simple et maintenable ;
2. respecte les conventions existantes du dépôt ;
3. documente brièvement la décision ;
4. évite de bloquer le développement pour un détail mineur.

À la fin de ton intervention, fournis un compte rendu indiquant :

- ce qui a été créé ;
- ce qui a été modifié ;
- les décisions techniques importantes ;
- les commandes exécutées ;
- les résultats du lint, du typecheck, des tests et du build ;
- les informations ou ressources encore nécessaires ;
- les prochaines étapes recommandées.

Commence maintenant par inspecter le dépôt, puis établis un plan d'implémentation concis avant de réaliser la première version fonctionnelle.